

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



HỌC SÂU VÀ ỨNG DỤNG
TRONG THỊ GIÁC MÁY TÍNH
Mã môn học: CO5085

BÁO CÁO BÀI TẬP E1–E3

Phân loại ảnh với CNN, attention
và mạng hồi quy

Học kỳ 261 · Năm học 2026–2027

Giảng viên:	Lê Thành Sách Nguyễn Quốc Minh
Sinh viên:	Huỳnh Thành Đạt
Mã số sinh viên:	2570161
Hình thức:	Cá nhân

Bản thảo triển khai — phần thực nghiệm chưa hoàn tất
TP. Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1	Dữ liệu và quy trình chung	1
1.1	Phạm vi bài làm	1
1.2	Tập dữ liệu và cách chia	1
1.3	Huấn luyện và chỉ số đánh giá	2
2	E1: Softmax, MLP và CNN	3
2.1	Ba bộ phân loại	3
2.2	Kết quả và phân tích lỗi	3
3	E2: MSA và cách lấy token	4
3.1	Hiện thực attention	4
3.2	Ba cách tokenization	4
3.3	So sánh cần thực hiện	5
4	E3: LSTM/GRU trên chuỗi ảnh	6
4.1	Biểu diễn ảnh và kiến trúc	6
4.2	Các phương trình được sử dụng	6
4.3	Kết quả và quan hệ không gian	7
5	Mở rộng E3: Điều kiện truyền gradient	8
5.1	Vấn đề và giả thuyết	8
5.2	Khởi tạo và regularization trực giao	8
5.3	Bias cổng nhớ theo độ dài chuỗi	9
5.4	Mean pooling và clipping	9
5.5	Ablation và cách kiểm tra	10
6	Tái lập và phần cần hoàn thiện	11
6.1	Mã nguồn và artifact	11
6.2	Phạm vi kết quả hiện có	11
6.3	Kết luận ở giai đoạn triển khai	12
	Ghi nhận hỗ trợ và liên chính học thuật	12
	Tài liệu tham khảo	13

Chương 1

Dữ liệu và quy trình chung

1.1 Phạm vi bài làm

Báo cáo trình bày ba bài tập E1–E3 trên cùng tập Fashion-MNIST. E1 so sánh softmax, MLP và CNN. E2 kiểm tra hiện thực multi-head self-attention (MSA) và ảnh hưởng của cách chia token. E3 xem ảnh như một chuỗi để phân loại bằng LSTM/GRU, đồng thời dùng lại MLP và CNN làm đối chứng. Đây là bài toán phân loại ảnh; “thời gian” của mạng hồi quy là thứ tự quét không gian, không phải thời gian trong video hay thứ tự từ trong câu.

Phần mở rộng tập trung vào LSTM: khởi tạo ma trận hồi quy trực giao, regularization trực giao mềm, khởi tạo bias cố định theo độ dài chuỗi và mean pooling. Mục đích là kiểm tra từng lựa chọn bằng ablation, không thay các yêu cầu bắt buộc bằng một kiến trúc phức tạp hơn. Tất cả trọng số đều học từ đầu.

Chưa huấn luyện. Các bảng chỉ có số tham số đếm từ kiến trúc; dấu – nghĩa là chưa đo.

Bản thảo hiện có thiết kế, công thức, mã nguồn và quy trình sinh kết quả. Kiểm thử số học không thay thế train trên dữ liệu thật. Các nhận xét về chất lượng mô hình sẽ chỉ được chốt sau khi có log. Hai bài tập lớn A1/A2 chưa nằm trong báo cáo vì chưa có nội dung đề tương ứng; không suy đoán yêu cầu của chúng từ E1–E3.

1.2 Tập dữ liệu và cách chia

Fashion-MNIST gồm ảnh thời trang mức xám 28×28 , 10 lớp, với 60.000 ảnh train và 10.000 ảnh test chính thức [2]. Chọn tập này vì có thể dùng nguyên kích thước ảnh cho cả ba bài, không cần backbone pretrained hoặc xử lý dữ liệu lớn.

Từ 60.000 ảnh train, tách phân tầng 10% làm validation với split seed 2026. Như vậy có 54.000 ảnh train, 6.000 validation và 10.000 test. Mỗi lớp lần lượt có 5.400, 600 và 1.000 ảnh theo phân bố chuẩn của bộ dữ liệu. Split seed không đổi giữa E1–E3 và không trùng vai trò với seed khởi tạo mô hình.

Ảnh được chuyển từ uint8 sang $[0, 1]$, sau đó chuẩn hóa $x' = (x - \mu)/\sigma$. Hai thống kê μ, σ chỉ tính trên 54.000 ảnh train, rồi giữ nguyên cho validation/test. Không dùng các giá trị chuẩn hóa chép từ một benchmark khác. Không augmentation trong nghiên cứu này để tránh thêm yếu tố gây nhiễu khi so sánh cách biểu diễn ảnh.

`common/data.py` lưu chỉ số từng split, checksum dữ liệu và fingerprint của cả split lẫn normalization. Loader ưu tiên HTTPS từ repo chính thức của dataset và giữ kiểm tra checksum của torchvision. Dữ liệu test có thể được tải cùng kho ảnh, nhưng không được đọc vào vòng

chọn checkpoint hay tính normalization.

1.3 Huấn luyện và chỉ số đánh giá

Mỗi mô hình xuất logits $z \in \mathbb{R}^{10}$. Với nhãn đúng y , cross-entropy là

$$\ell(z, y) = -z_y + \log \sum_{k=1}^{10} \exp(z_k), \quad \frac{\partial \ell}{\partial z_k} = p_k - \mathbf{1}[k = y]. \quad (1.1)$$

Vì `cross_entropy` đã thực hiện log-softmax ổn định số, code không chèn softmax trước loss. Softmax chỉ được dùng khi cần xác suất để trình bày dự đoán.

Thiết lập	Giá trị định trước
Optimizer	AdamW, weight decay 10^{-4} trên các tham số trainable
Ngân sách	30 epoch, batch 256, không early stopping
Learning rate	E1/E3: 10^{-3} ; E2: 3×10^{-4}
Scheduler	Cosine decay về 1% learning rate ban đầu
Gradient clipping	Global norm tối đa 1, sau backward và trước step
Độ chính xác số	FP32; tắt AMP, TF32 và compile
Seed mô hình	Dự kiến 42, 43, 44; cùng split seed 2026
Chọn checkpoint	Validation accuracy lớn nhất; hòa thì CE nhỏ nhất

Bảng 1.1: Protocol chung; đây là cấu hình, không phải kết quả đo.

Vòng lặp được viết trực tiếp: zero gradient, forward, tính loss, backward, clipping, optimizer step. Chỉ đánh giá test bằng một lệnh riêng sau khi chốt thí nghiệm. Khi loss hoặc gradient không hữu hạn, chương trình dừng thay vì thay bằng một số hợp lệ giả.

Accuracy là chỉ số chính; macro-F1 bổ sung khi phân tích lỗi theo lớp. Confusion matrix và các dự đoán sai có confidence cao giúp chỉ ra cặp lớp cần xem lại. Số tham số biểu thị một phần capacity, không đồng nghĩa với FLOPs hay khả năng tổng quát hóa. Thời gian train/epoch được đồng bộ CUDA, gồm bước lấy batch và cập nhật; bỏ epoch đầu khi lấy median. Tốc độ inference được đo riêng, bỏ hai batch đầu và không tính truyền dữ liệu sang GPU. Chỉ so tốc độ khi môi trường và cách đo tương đương.

Bảng kết quả tổng hợp theo seed thực có. Độ lệch chuẩn mẫu chỉ xác định khi có ít nhất hai seed và không phải khoảng tin cậy. Các hình đại diện dùng seed nhỏ nhất, không chọn seed có test tốt nhất. Đây là phép so sánh dưới một ngân sách công bố trước, chưa phải cuộc tìm kiếm siêu tham số tối ưu cho mọi họ mô hình.

Chương 2

E1: Softmax, MLP và CNN

2.1 Ba bộ phân loại

Softmax nhận vector $x \in \mathbb{R}^{784}$ và tính logits $z = Wx + b$; ranh giới giữa hai lớp là siêu phẳng trong không gian pixel. MLP bổ sung hai tầng phi tuyến trên cùng vector flatten. CNN dùng kernel chia sẻ theo vị trí để học đặc trưng cục bộ. Ba kiến trúc được giữ nhỏ và mô tả đầy đủ trong bảng dưới đây.

Mô hình	Các lớp theo thứ tự
Softmax	Flatten $784 \rightarrow 10$
MLP	Flatten; $784 \rightarrow 256$; ReLU; dropout 0,2; $256 \rightarrow 128$; ReLU; dropout 0,2; $128 \rightarrow 10$
CNN	Conv 1 $\rightarrow 32$, kernel 3, padding 1; BN; ReLU; pool 2; Conv 32 $\rightarrow 64$ tương tự; pool 2; flatten; $3136 \rightarrow 128$; ReLU; dropout 0,2; $128 \rightarrow 10$

Bảng 2.1: Kiến trúc E1. Conv không dùng bias vì sau đó có batch normalization.

Hai phép pooling đưa ảnh $28 \rightarrow 14 \rightarrow 7$, vì vậy head CNN nhận $64 \times 7 \times 7 = 3136$ giá trị. Với một lớp Linear $d_{in} \rightarrow d_{out}$ có bias, số tham số là $d_{in}d_{out} + d_{out}$. Ví dụ softmax có $784 \times 10 + 10 = 7.850$ tham số. Các số còn lại được đếm trực tiếp bằng `numel()`, gồm tham số affine của BN nhưng không gồm running statistics.

2.2 Kết quả và phân tích lỗi

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
Softmax	7,850	—	—	—
MLP	235,146	—	—	—
CNN	421,738	—	—	—

Bảng 2.2: Kết quả E1; chỉ số chưa đo để trống.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; —: chưa đo; $\pm$: SD qua seed. Phạm vi seed thực tế ở Chương 6; cách đo thời gian ở Chương 1.

Cần đọc đồng thời train/validation CE và accuracy để phân biệt dấu hiệu overfit với mô hình chưa học đủ. Không xếp hạng khả năng tổng quát hóa chỉ bằng số tham số. Confusion matrix sẽ chỉ ra cặp lớp cần kiểm tra lại trên ảnh gốc; chưa có log thì chưa xác nhận cặp nào dễ nhầm nhất.

Chương 3

E2: MSA và cách lấy token

3.1 Hiện thực attention

Với tensor token $X \in \mathbb{R}^{B \times N \times d}$, mỗi head học các phép chiếu query, key và value. Bỏ chỉ số batch để gọn, attention của một head là [3]

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad A = \text{softmax}_{\text{hàng}} \left(\frac{QK^\top}{\sqrt{d_h}} \right), \quad O = AV. \quad (3.1)$$

Các bias được lược bỏ trong công thức cho gọn, nhưng có trong code. Ở đây $d = 64$, có bốn head nên $d_h = 16$. Code gộp ba phép Linear thành QKV, reshape thành các head, nhân ma trận, softmax theo chiều key, áp dụng dropout, ghép head và chiếu output. Bản manual không gọi `nn.MultiheadAttention`, `TransformerEncoder` hoặc fused attention để thay phần tính này.

Bản đối chiếu dùng `nn.MultiheadAttention(batch_first=True)`. Chương trình tạo toàn bộ model manual rồi copy đúng QKV và output projection sang module PyTorch. Các phần tokenizer, positional embedding, FFN, LayerNorm và head cũng giữ nguyên tensor khởi tạo của cùng seed. Chỉ reset seed trước hai constructor khác nhau là chưa đủ bảo đảm điều này.

Kiểm thử dùng tensor nhỏ và dropout bằng 0: so output, gradient input và gradient của tất cả tham số. Kiểm thử này kiểm tra tính tương đương số học, không đánh giá mô hình đã học. Trong training, dropout và kernel có thể khác cách tiêu thụ số ngẫu nhiên hoặc thứ tự tính, nên không yêu cầu hai log phải giống từng bit.

3.2 Ba cách tokenization

Cách chia	N	Đầu vào/token	Phép tạo embedding
Hàng	28	28	Linear $28 \rightarrow 64$
Patch 4×4	49	16	Unfold không chồng lấp, Linear $16 \rightarrow 64$
CNN-stem	49	64	Hai Conv stride 2: $1 \rightarrow 32 \rightarrow 64$, BN/ReLU; flatten spatial

Bảng 3.1: Tokenization của E2 trên ảnh $1 \times 28 \times 28$.

Token hàng gom các pixel cùng một hàng. Token patch quét lưới từ trái sang phải, trên xuống dưới. CNN-stem rút feature map $64 \times 7 \times 7$; mỗi vị trí là một token đã qua xử lý cục bộ. Stem được train từ đầu, không phải CNN pretrained. Stem có thêm phép tính và tham số, vì vậy nếu nó tốt hơn thì không thể quy toàn bộ lợi ích cho số token.

Mỗi cấu hình thêm learned positional embedding, qua hai encoder block dạng pre-LN và kết thúc bằng LayerNorm, mean pooling và Linear $64 \rightarrow 10$. Không có CLS token. Mỗi block tính

$$U = X + \text{Dropout}(\text{MSA}(\text{LN}(X))), \quad (3.2)$$

$$Y = U + \text{FFN}(\text{LN}(U)), \quad (3.3)$$

trong đó FFN là $64 \rightarrow 128 \rightarrow 64$ với GELU và dropout 0,1. Attention không dùng causal mask vì toàn bộ ảnh đã có tại thời điểm phân loại.

3.3 So sánh cần thực hiện

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
patches / manual	71,946	—	—	—
patches / torch	71,946	—	—	—
rows / manual	71,370	—	—	—
rows / torch	71,370	—	—	—
cnn / manual	89,770	—	—	—
cnn / torch	89,770	—	—	—

Bảng 3.2: Sáu cấu hình E2. Manual và PyTorch của cùng tokenizer có cùng số tham số.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; —: chưa đo; $\pm$: SD qua seed. Phạm vi seed thực tế ở Chương 6; cách đo thời gian ở Chương 1.

Trước hết so manual với thư viện ở từng tokenizer để kiểm tra tác động của cách hiện thực. Sau đó so ba tokenizer khi giữ attention backend cố định. Chi phí phần score của attention tỉ lệ $N^2 d$; riêng số cặp token của patch so với hàng là $49^2/28^2 \approx 3,06$. Tỉ lệ này không phải tỉ lệ thời gian toàn mô hình vì còn chi phí projection, FFN, stem và kernel.

Báo cáo cần đối chiếu accuracy với thời gian và capacity, không chỉ chọn dòng accuracy lớn nhất. Manual chậm hơn chưa có nghĩa là sai; thư viện có thể dùng kernel tối ưu hơn. Ngược lại, sai lệch lớn về điểm số cần kiểm tra split, token order, khởi tạo và quy trình train trước khi diễn giải thành ưu thế của một API.

Chương 4

E3: LSTM/GRU trên chuỗi ảnh

4.1 Biểu diễn ảnh và kiến trúc

Ảnh được biến thành chuỗi theo hai cách đã dùng trong E2, nhưng không qua embedding học được: hàng cho $T = 28, D = 28$; patch 4×4 cho $T = 49, D = 16$. Cả LSTM và GRU đều có một tầng, một chiều, hidden size $H = 128$, trạng thái đầu bằng zero. Baseline lấy hidden state cuối, dropout 0,1 rồi Linear $128 \rightarrow 10$.

Quét hàng giữ các quan hệ ngang trong một token và để mạng hồi quy xử lý quan hệ giữa các hàng. Quét patch tạo nhiều bước hơn, mỗi bước mô tả một vùng nhỏ hơn. Hai cách thay đổi đồng thời độ dài chuỗi và kích thước input; không nên gọi đây là thí nghiệm chỉ thay T .

4.2 Các phương trình được sử dụng

Với LSTM, đặt $a_t = W_x x_t + W_h h_{t-1} + b$ rồi tách thành bốn phần [6]:

$$i_t = \sigma(a_t^i), \quad f_t = \sigma(a_t^f), \quad g_t = \tanh(a_t^g), \quad o_t = \sigma(a_t^o), \quad (4.1)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t). \quad (4.2)$$

c_t có đường cập nhật cộng, còn h_t là trạng thái dùng cho output và hồi tiếp. Cổng quên lớn có thể giữ thông tin, nhưng kiến trúc này không tự bảo đảm mọi gradient đều được bảo toàn.

GRU dùng reset gate r_t , update gate z_t và trạng thái ứng viên n_t . Diagnostic khớp đúng cách PyTorch đặt reset gate sau phép chiếu hidden [7]:

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr}), \quad (4.3)$$

$$z_t = \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz}), \quad (4.4)$$

$$n_t = \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn})), \quad (4.5)$$

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}. \quad (4.6)$$

Cách đặt r_t này không được tự ý thay bằng $W_{hn}(r_t \odot h_{t-1})$ khi đối chiếu với module thư viện. GRU không có một cell state riêng như LSTM.

PyTorch lưu hai vector bias cho mỗi tầng. Vì vậy, số tham số phần recurrent là $4H(D + H + 2)$ với LSTM và $3H(D + H + 2)$ với GRU; cộng thêm $10H + 10$ cho head. Giữ cùng H giúp so sánh rõ nhưng không làm hai mô hình có cùng capacity.

4.3 Kết quả và quan hệ không gian

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
MLP	235,146	—	—	—
CNN	421,738	—	—	—
LSTM / rows	82,186	—	—	—
GRU / rows	61,962	—	—	—
LSTM / patches	76,042	—	—	—
GRU / patches	57,354	—	—	—

Bảng 4.1: E3 và hai baseline từ E1, trên cùng split.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; —: chưa đo; $\pm$: SD qua seed. Phạm vi seed thực tế ở Chương 6; cách đo thời gian ở Chương 1.

CNN khai thác lân cận hai chiều và chia sẻ kernel; LSTM/GRU tích lũy thông tin theo đường quét. Khi chuyển từ cuối một hàng patch sang đầu hàng kế tiếp, hai phần tử kề nhau trong chuỗi chưa chắc là hàng xóm gần nhất trên ảnh. Đây là đặc điểm của biểu diễn, không phải lỗi của cell.

Ngoài accuracy và số tham số, cần xem ảnh hưởng của cách quét và gradient theo vị trí. E3.diagnose dùng validation để so thứ tự gốc, đảo ngược và một hoán vị cố định. Mô hình không được train lại khi đảo thứ tự. Suy giảm accuracy trong phép thử này phản ánh sự nhạy với thay đổi phân bố đầu vào; riêng nó chưa chứng minh mạng hiệu phụ thuộc dài hạn.

Chương 5

Mở rộng E3: Điều kiện truyền gradient

5.1 Vấn đề và giả thuyết

Chuỗi hàng/patch chỉ dài 28 hoặc 49 bước. Độ dài này chưa phải một benchmark long-context, và chưa có số đo để kết luận baseline đang bị vanishing gradient nghiêm trọng. Tuy vậy, nó đủ để xây một thí nghiệm có kiểm soát về khởi tạo, phép đọc trạng thái và đường truyền gradient. Các thay đổi dưới đây đều được hiện thực, không chỉ đề xuất bằng lý thuyết.

Với trạng thái hồi quy tổng quát $s_t = F_\theta(s_{t-1}, x_t)$ và $J_t = \partial s_t / \partial s_{t-1}$, đóng góp từ loss cuối chuỗi thỏa

$$\frac{\partial \ell_T}{\partial s_t} = J_{t+1}^\top J_{t+2}^\top \cdots J_T^\top \frac{\partial \ell_T}{\partial s_T}, \quad \left\| \frac{\partial \ell_T}{\partial s_t} \right\| \leq \prod_{k=t+1}^T \|J_k\|_2 \left\| \frac{\partial \ell_T}{\partial s_T} \right\|. \quad (5.1)$$

Tích nhiều phép co có thể làm gradient nhỏ; tích nhiều phép khuếch đại có thể làm nó lớn. Cận theo norm là một cách phân tích, không mô tả chính xác mọi hướng gradient. Với LSTM, s_t cần gồm cả h_t, c_t , không chỉ một biến hidden [4].

5.2 Khởi tạo và regularization trực giao

Với từng gate, ma trận hồi quy vuông $W_g \in \mathbb{R}^{H \times H}$ được khởi tạo trực giao ở các cấu hình tương ứng. Nếu $W_g^\top W_g = I$ thì $\|W_g v\|_2 = \|v\|_2$; riêng phép biến đổi tuyến tính đó không làm thay đổi norm. Điều này không loại bỏ tác động của sigmoid, tanh, các gate và tích Jacobian toàn cell.

Tập các ma trận trực giao là trường hợp vuông của Stiefel manifold. Thay vì dùng optimizer có ràng buộc trên đa tạp, code dùng một penalty mềm trong không gian tham số thông thường:

$$\mathcal{R}_{orth} = \frac{1}{GH} \sum_{g=1}^G \|W_g^\top W_g - I\|_F^2, \quad \mathcal{L} = \mathcal{L}_{CE} + \lambda \mathcal{R}_{orth}, \quad (5.2)$$

trong đó $G = 4$ cho LSTM, $H = 128$ và $\lambda = 10^{-3}$ ở các preset có regularization. Tách theo gate là cần thiết; không coi ma trận ghép $4H \times H$ là một khối rồi gọi mỗi gate đã trực giao.

Đặt $A = W^\top W - I = A^\top$. Vì phân cho một gate là

$$d \|A\|_F^2 = 2 \operatorname{tr}(A dA) = 2 \operatorname{tr}[A(dW^\top W + W^\top dW)] \quad (5.3)$$

$$= 4 \operatorname{tr}[(WA)^\top dW], \quad (5.4)$$

do đó

$$\nabla_{W_g} \mathcal{R}_{orth} = \frac{4}{GH} W_g (W_g^\top W_g - I). \quad (5.5)$$

Công thức được kiểm tra với autograd. Penalty tạo chi phí tính toán thêm và có thể làm giảm mức linh hoạt của nghiệm. AdamW vẫn có weight decay cố định trên mọi tham số, nên nó đồng thời kéo trọng số về zero; không được mô tả quá trình train như duy trì trực giao chính xác.

Đây là penalty mềm trong không gian Euclid, không dùng diffusion maps, đồ thị dữ liệu hoặc optimizer manifold.

5.3 Bias cổng nhớ theo độ dài chuỗi

Ở cấu hình chrono, đặt $b_f = \log(T - 1)$, $b_i = -\log(T - 1)$; bias các gate khác bằng zero. Chỉ thay bias_ih ; giữ $\text{bias_hh}=0$ để tổng bias đúng giá trị dự định. Đây là biến thể *một timescale cố định* lấy cảm hứng từ chrono initialization, không phải chép nguyên cách lấy timescale ngẫu nhiên trong công trình gốc [5].

Khi bỏ qua đóng góp từ input/hidden tại lúc khởi tạo,

$$f_t \approx \sigma(\log(T - 1)) = 1 - \frac{1}{T}, \quad i_t \approx \frac{1}{T}. \quad (5.6)$$

Giữ các gate cố định để chỉ xét đường trực tiếp $c_{t-1} \rightarrow c_t$, ta có

$$\left. \frac{\partial c_T}{\partial c_t} \right|_{\text{đường trực tiếp}} = \prod_{k=t+1}^T \text{diag}(f_k). \quad (5.7)$$

Nếu mỗi $f_k \approx 1 - 1/T$ thì độ suy giảm trên khoảng T bước gần $(1 - 1/T)^T \approx e^{-1}$, thay vì 2^{-T} khi $f_k \approx 0,5$. Đây chỉ là xấp xỉ để giải thích khởi tạo. Đạo hàm tổng còn có các đường qua hidden và đạo hàm của gate; không thể đồng nhất nó với tích trên.

Cổng quên gần 1 đi kèm cổng nhập nhỏ có thể khiến mô hình cập nhật thông tin mới chậm. Bias lớn cũng có thể đẩy sigmoid về vùng bão hòa. Vì vậy bias được học tiếp trong train, không bị khóa, và chrono cần một đối chứng riêng thay vì giả định luôn có lợi.

5.4 Mean pooling và clipping

Baseline dự đoán từ h_T . Biến thể mean pooling dùng $\bar{h} = T^{-1} \sum_t h_t$ trước head. Bỏ qua dropout trong biểu thức minh họa, nếu $z = W_o \bar{h} + b$ và y là vector one-hot thì đường gradient trực tiếp tới mỗi trạng thái là

$$\left. \frac{\partial \ell}{\partial h_t} \right|_{\text{head}} = \frac{1}{T} W_o^\top (p - y). \quad (5.8)$$

Đạo hàm tổng còn gồm các đường hồi quy về sau. Lợi ích tiềm năng là một trạng thái sớm không chỉ phụ thuộc vào việc tín hiệu đi qua toàn bộ chuỗi để tới head. Đối lại, trung bình có thể làm mờ vị trí và trộn vùng nền vào biểu diễn. Learned position không được thêm cho E3;

mạng hồi quy tự phụ thuộc thứ tự quét.

Clipping global norm được dùng cho tất cả baseline lẫn cải tiến:

$$g \leftarrow g \min\left(1, \frac{\tau}{\|g\|_2}\right), \quad \tau = 1. \quad (5.9)$$

Nó giới hạn gradient quá lớn trước bước cập nhật, không khuếch đại gradient đã biến mất. Không gọi clipping là lời giải cho vanishing gradient [4].

5.5 Ablation và cách kiểm tra

Các cấu hình dưới đây chỉ dùng LSTM theo hàng, giữ $H = 128$, split và budget. “Orth init” chỉ thay khởi tạo. “Orth reg” thêm penalty vào bản orth init. “Combined” dùng trực giao ban đầu, penalty, chrono và mean pooling. Hai bản “no reg” và “no mean” bỏ riêng một thành phần từ combined; các thành phần khác giữ nguyên.

Mô hình	Tham số	Val (%)	Test (%)	Giây/epoch
LSTM / rows	82,186	—	—	—
LSTM + orthogonal init	82,186	—	—	—
LSTM + orthogonal init/reg	82,186	—	—	—
LSTM + chrono bias	82,186	—	—	—
LSTM + mean pooling	82,186	—	—	—
LSTM combined	82,186	—	—	—
Combined without regularizer	82,186	—	—	—
Combined without mean pooling	82,186	—	—	—

Bảng 5.1: Baseline và bảy cấu hình mở rộng. Tất cả có cùng số tham số trainable.

Val/Test: accuracy (%). Tham số đếm từ kiến trúc; —: chưa đo; $\pm$: SD qua seed. Phạm vi seed thực tế ở Chương 6; cách đo thời gian ở Chương 1.

Chênh lệch orth init–baseline xét khởi tạo; orth reg–orth init xét penalty. Chrono và mean được so riêng với baseline. So combined với hai ablation cho biết tác động trong tổ hợp, nhưng chưa phải thiết kế factorial đầy đủ và chưa tách mọi tương tác. Không dùng việc chỉ một seed tăng để khẳng định cải tiến bền vững.

Diagnostic unroll dùng đúng trọng số của checkpoint native và công thức cell tương ứng, giữ gradient tại từng trạng thái trung gian thật. Loss dùng tổng theo batch để gradient từng mẫu không bị chia bởi batch size; sau đó lấy trung bình norm qua mẫu. Gradient của tensor output từ một fused RNN không thay thế được phép đo BPTT-state này.

Trên cùng 256 ảnh validation theo quy tắc định trước, ghi $\|\partial\ell/\partial h_t\|$, $\|\partial\ell/\partial c_t\|$, gradient theo input token, giá trị gate trung bình và singular values của từng W_g . Gradient nhỏ ở pixel nền không nhất thiết là lỗi học; cần đọc cùng accuracy, CE và ảnh. Chưa có các phép đo này thì chưa thể kết luận baseline bị vanishing hay tổ hợp đã khắc phục được nó.

Chương 6

Tái lập và phần cần hoàn thiện

6.1 Mã nguồn và artifact

Repo tách mô hình thành E1/, E2/, E3/; phần dùng chung nằm trong common/. Mỗi run có run.json, history.csv, best.pt, last.pt và metrics.json. Đánh giá test sinh thêm prediction CSV, confusion matrix, ảnh sai và chỉ số theo lớp. Hyperparameter được khai báo trong một file JSON gồm 20 preset.

Checkpoint cuối epoch lưu model, AdamW, cosine scheduler, lịch sử và trạng thái RNG của Python, NumPy, PyTorch, CUDA, shuffle generator. Resume từ epoch hoàn tất gần nhất, không từ batch đang dở. Fingerprint kiểm tra code, config, seed, dữ liệu và normalization; môi trường torch/CUDA cũng được đối chiếu. Khóa thư mục ngăn hai tiến trình ghi cùng run. Cơ chế này không hứa khả năng tái lập bitwise giữa các phần cứng khác nhau.

scripts/export_report.py chỉ gộp các run đúng protocol. Nó xuất bảng LaTeX, CSV/JSON và sao chép hình từ log thật; không tự tạo số liệu khi thiếu file. Khi chỉ có một phần thí nghiệm, bảng seed bên dưới thể hiện đúng phần đó. Những đoạn phân tích trong bản thảo không được tự động biến thành kết luận thành tích.

6.2 Phạm vi kết quả hiện có

Chưa huấn luyện. Các bảng chỉ có số tham số đếm từ kiến trúc; dấu – nghĩa là chưa đo.

Chưa có seed nào hoàn tất. Nghiên cứu có 20 cấu hình; kế hoạch ba seed 42, 43, 44 tương ứng 60 lượt. Danh sách cấu hình nằm trong configs/experiments.json; đây không phải số lượt đã chạy.

Các phép kiểm tra số học hiện có bao gồm tính tương đương forward/gradient của MSA, token order, unroll LSTM/GRU, đạo hàm regularizer, bias chrono, split, khôi phục shuffle và artifact không có số liệu. Kiểm thử chạy trên CPU; lượt huấn luyện đầy đủ và phép đo CUDA còn phải thực hiện trên máy đích.

Sau khi có kết quả, cần hoàn thiện nhận xét từng chương theo cùng câu hỏi đã đặt ra: khác biệt E1 có đi kèm overfit hay không; tokenizer nào cân bằng accuracy và chi phí; LSTM/GRU có cạnh tranh với CNN trên cùng ảnh hay không; thành phần mở rộng nào đóng góp và có ổn định theo seed hay không. Nếu improvement không tốt hơn, vẫn giữ số liệu và giới hạn đó trong báo cáo.

6.3 Kết luận ở giai đoạn triển khai

Ba bài đã có hiện thực riêng cho các họ mô hình và một quy trình dữ liệu/đánh giá thống nhất. Phần mở rộng được chuyển thành những thay đổi cụ thể trong LSTM, có công thức, đối chứng và công cụ đọc gradient. Điều có thể xác nhận ở giai đoạn này là cấu trúc triển khai và các kiểm tra số học; chưa thể xếp hạng mô hình theo chất lượng hoặc tuyên bố cải tiến làm tăng accuracy.

Ghi nhận hỗ trợ và liên chính học thuật

Bản mã nguồn và văn bản ban đầu có mức hỗ trợ AI đáng kể, gồm hiện thực, kiểm thử và soạn thảo. Đề môn học chỉ cho phép AI hỗ trợ và không cho nộp sản phẩm do AI tạo phần lớn. Bản này cần được người học tự kiểm tra, phát triển lại và hoàn thiện bằng thực nghiệm của mình, đồng thời làm rõ phạm vi hỗ trợ được giảng viên chấp nhận; không nộp nguyên trạng như sản phẩm hoàn toàn tự thực hiện.

Bố cục LaTeX dựa trên mẫu Diffusion Maps được cung cấp. Repo tham khảo chỉ được xem về tổ chức thư mục; không lấy code, checkpoint, bảng, hình hoặc kết quả thực nghiệm của bài mẫu.

Tài liệu tham khảo

- [1] Lê Thành Sách, Nguyễn Quốc Minh. *Bài tập E1–E3: Học sâu và ứng dụng trong thị giác máy tính*, CO5085. Đề học kỳ 261 (2026–2027), file `exercise-vne.pdf` được cung cấp.
- [2] H. Xiao, K. Rasul, R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747, 2017. <https://arxiv.org/abs/1708.07747>. Dữ liệu: <https://github.com/zalandoresearch/fashion-mnist>.
- [3] A. Vaswani và cộng sự. *Attention Is All You Need*. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>.
- [4] R. Pascanu, T. Mikolov, Y. Bengio. *On the difficulty of training recurrent neural networks*. ICML, 2013; arXiv:1211.5063. <https://arxiv.org/abs/1211.5063>.
- [5] C. Tallec, Y. Ollivier. *Can recurrent neural networks warp time?* ICLR, 2018. <https://arxiv.org/abs/1804.11188>.
- [6] PyTorch. *LSTM: API and equations*. <https://docs.pytorch.org/docs/2.9/generated/torch.nn.LSTM.html>.
- [7] PyTorch. *GRU: API and implementation notes*. <https://docs.pytorch.org/docs/2.9/generated/torch.nn.GRU.html>.
- [8] PyTorch. *Previous versions: torch 2.10.0 / torchvision 0.25.0*. <https://pytorch.org/get-started/previous-versions/>. Cập phiên bản ghim cho môi trường chạy; không khẳng định đây là bản mới nhất.